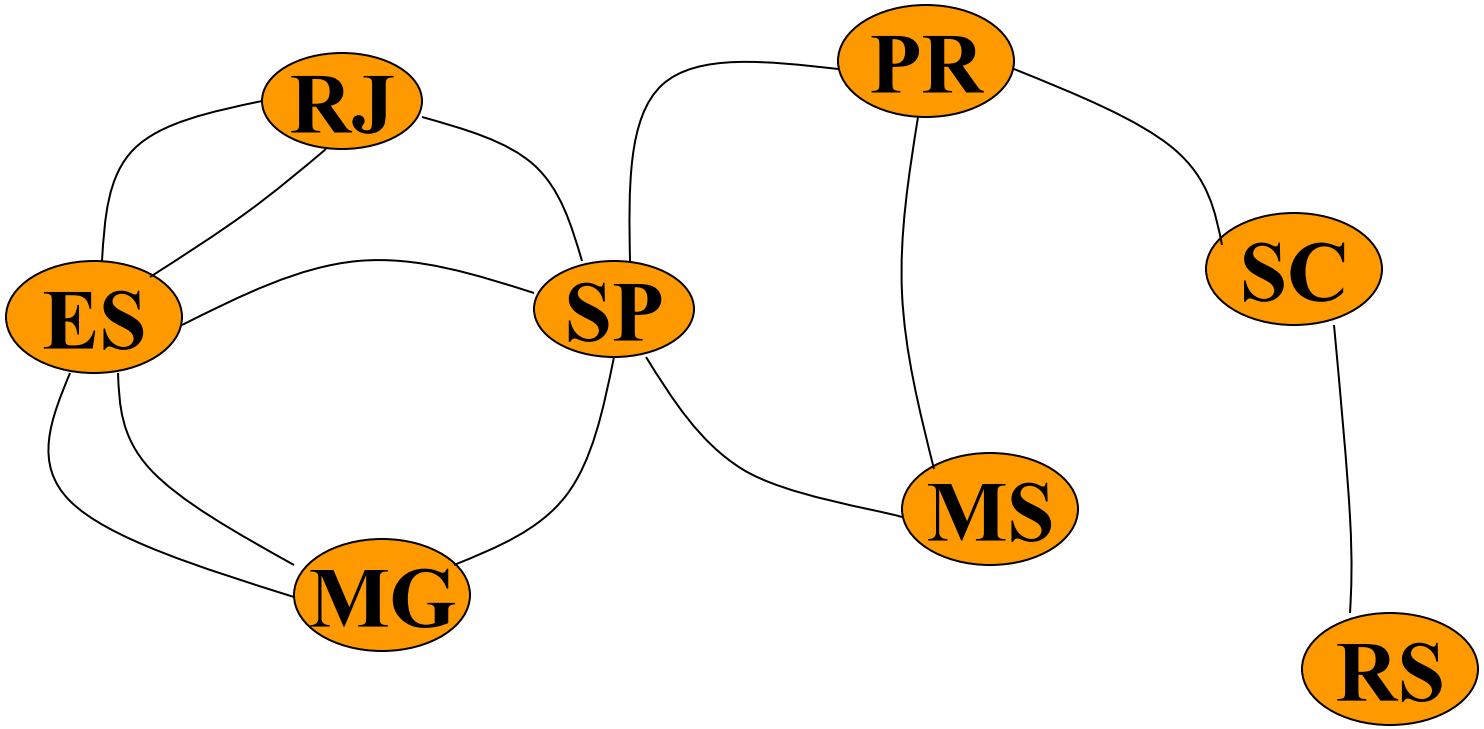
Teoria dos Grafos

Implementação – Matriz de Adjacência e Exercícios



Material Teórico do prof. Dr. Ivan Carlos Alcântara de Oliveira
e Implementação baseada no Material do prof. Ms. Fábio Lubacheski

Grafos - Representação - Matriz de Adjacência (1/8)

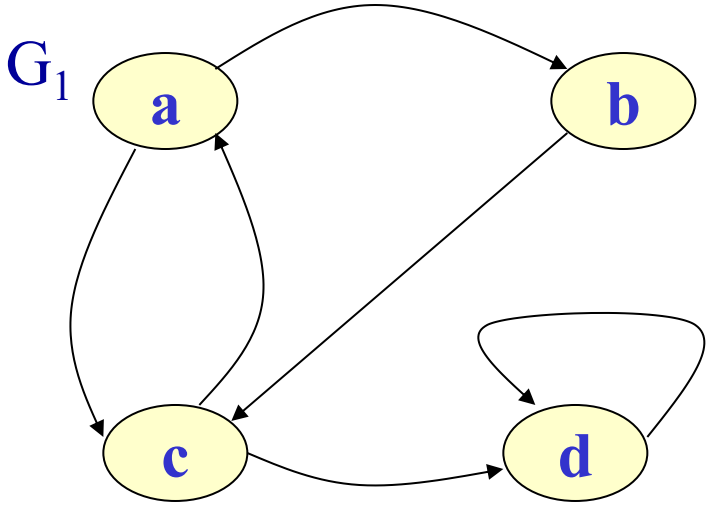
- Considere um **grafo direcionado G = (V, E)** com n vértices, V = { v_1, v_2, v_3, ..., v_n }. Um esquema simples para representá-lo é através de uma **matriz de adjacência $A_{n \times n}$** de **0s e 1s** dada por:

$$A_{i,j} = \begin{cases} \textbf{1 se } (v_i, v_j) \in E \\ \textbf{0 caso contrário} \end{cases}$$

- Isto é, o (i, j)-ésimo elemento da matriz é 1 somente se $v_i \rightarrow v_j$ é uma aresta de G.

Grafos - Representação - Matriz de Adjacência (2/8)

Exemplo:



$A_1 =$

	a	b	c	d
a	0	1	1	0
b	0	0	1	0
c	1	0	0	1
d	0	0	0	1

Grafos - Representação - Matriz de Adjacência (3/8)

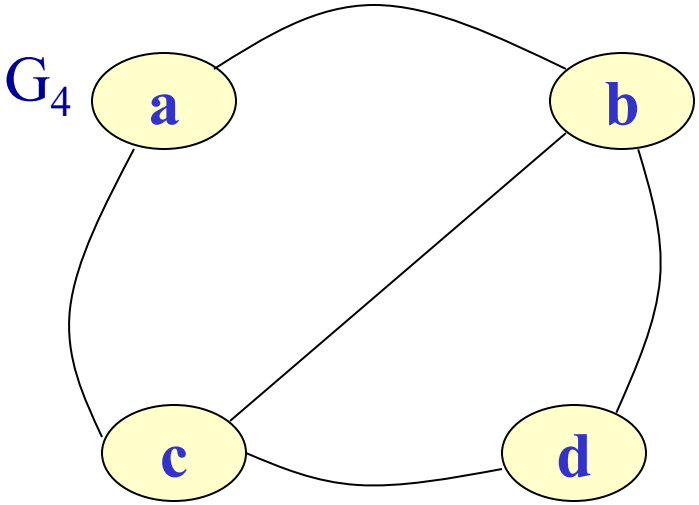
- Considere um **grafo não-direcionado G = (V, E)** com n vértices, V = { v_1, v_2, v_3, ..., v_n }. Um esquema simples para representá-lo é através de uma **matriz de adjacência $A_{n \times n}$** de **0s e 1s** dada por:

$$A_{i,j} = \begin{cases} \textbf{1 se } \{v_i, v_j\} \in E \\ \textbf{0 caso contrário} \end{cases}$$

- Os conjuntos { v_i, v_j } e {v_j, v_i } são equivalentes, assim, a matriz é simétrica em relação a diagonal.

Grafos - Representação - Matriz de Adjacência (4/8)

Exemplo:



$A_4 =$

	a	b	c	d
a	0	1	1	0
b	1	0	1	1
c	1	1	0	1
d	0	1	1	0

Grafos

Implementação – Grafo Direcionado – Matriz de Adjacência*

- Para representar um grafo direcionado como uma matriz de adjacência vamos definir uma classe com os seus atributos e métodos, como a seguir:

```
class TGrafo{
      private:
            int n; // quantidade de vértices
            int m; // quantidade de arestas
            int **adj; //matriz de adjacência
      public:
            TGrafo( int n);
            void insereA(int v, int w);
            void removeA(int v, int w);
            void show();
            ~TGrafo();
};
```

Baseado na implementação do prof. Ms. Fábio Lucacheski

Grafos

Implementação – Grafo Direcionado – Matriz de Adjacência*

```
// Construtor do TGrafo, responsável por
// Criar a matriz de adjacência v x v do Grafo
TGrafo::TGrafo( int n ){
    this->n = n;
    // No início dos tempos não há arestas
    this->m = 0;
    // aloca da matriz do TGrafo
    int **adjac = new int*[n];
    for(int i = 0; i < n; i++)
            adjac[i]= new int[n];
    adj = adjac;
    // Inicia a matriz com zeros
     for(int i = 0; i< n; i++)
            for(int j = 0; j< n; j++)
                    adj[i][j]=0;
}
```

Grafos

Implementação – Grafo Direcionado – Matriz de Adjacência*

```
// Destructor, responsável por
// liberar a memória alocada para a matriz
TGrafo::~TGrafo(){
        n = 0;
        m = 0;
        delete [] *adj;
        cout << "espaço liberado";
}
```

Grafos

Implementação – Grafo Direcionado – Matriz de Adjacência*

```
// Insere uma aresta no Grafo tal que
// v é adjacente a w
void TGrafo::insereA( int v, int w){
    // testa se nao temos a aresta
    if(adj[v][w] == 0 ){
        adj[v][w] = 1;
        m++; // atualiza qtd arestas
    }
}
```

Grafos

Implementação – Grafo Direcionado – Matriz de Adjacência*

```
// remove uma aresta v->w do Grafo
void TGrafo::removeA(int v, int w){
    // testa se temos a aresta
    if(adj[v][w] == 1 ){
        adj[v][w] = 0;
        m--; // atualiza qtd arestas
    }
}
```

Grafos

Implementação – Grafo Direcionado – Matriz de Adjacência*

```
// remove uma aresta v->w do Grafo
void TGrafo::removeA(int v, int w){
    // testa se temos a aresta
    if(adj[v][w] == 1 ){
        adj[v][w] = 0;
        m--; // atualiza qtd arestas
    }
}
```

Grafos

Implementação – Grafo Direcionado – Matriz de Adjacência*

```
// número de vértices, arestas e a matriz de adjacência obtida
void TGrafo::show(){
   cout << “n: " << n << endl;
   cout << “m: " << m << endl;
   for( int i=0; i < n; i++){
      cout << "\n";
      for( int w=0; w < n; w++)
         if(adj[i][w] == 1)
            cout << "Adj[" << i<< "," << w << "]= 1" << " ";
         else cout << "Adj[" << i<< "," << w << "]= 0" << " ";
   }
   cout <<"\nfim da impressao do grafo." << endl;
}
```

Grafos

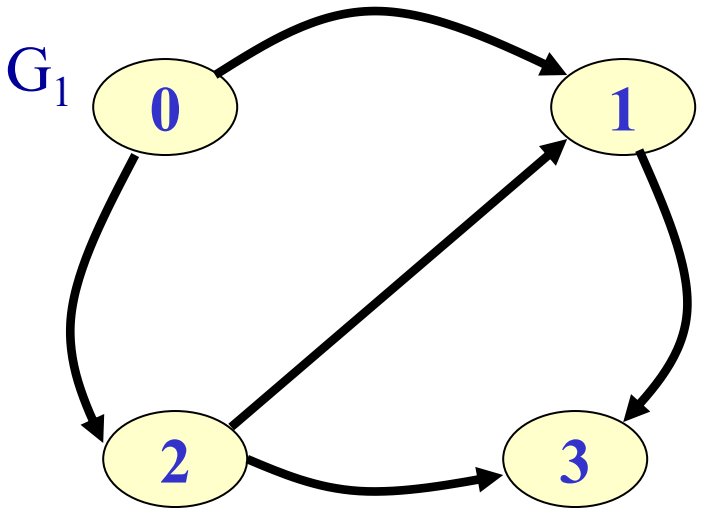
Implementação – Grafo Direcionado – Matriz de Adjacência*

```
// número de vértices, arestas e a matriz de adjacência obtida
void TGrafo::show(){
    cout << "\nV: " << v << endl;
    cout << "A: " << a << endl;
    for( int i=0; i < v; i++){
        cout << "\n";
        for( int w=0; w < v; w++)
            if(adj[i][w] == 1)
                cout << "Adj[" << i<< "," << w << "]= 1" << " ";
            else cout << "Adj[" << i<< "," << w << "]= 0" << " ";
    }
    cout <<"\nfim da impressao do grafo." << endl;
}
```

Grafos

Implementação – Grafo Direcionado – Matriz de Adjacência*

```
// Programa responsável por manipular o Grafo
int main(){
    //  chama o construtor para criar um grafo 4x4
    TGrafo g(4);
    //insere as arestas do grafo
    //A={(0,1),(0,2),(2,1),(2,3),(1,3)}
    g.insereA(0,1);
    g.insereA(0,2);
    g.insereA(2,1);
    g.insereA(2,3);
    g.insereA(1,3);
    // mostra o grafo preenchido
    g.show();
    return 0;
}
```




```
C:\data\_Mackenzie\2022\1oSemestre\TG_ccp_T...
n: 4
m: 5

Adj[0,0]= 0 Adj[0,1]= 1 Adj[0,2]= 1 Adj[0,3]= 0
Adj[1,0]= 0 Adj[1,1]= 0 Adj[1,2]= 0 Adj[1,3]= 1
Adj[2,0]= 0 Adj[2,1]= 1 Adj[2,2]= 0 Adj[2,3]= 1
Adj[3,0]= 0 Adj[3,1]= 0 Adj[3,2]= 0 Adj[3,3]= 0
fim da impressao do grafo.
espaco liberado
--------------------------------
Process exited after 5.762 seconds with return value 0
Press any key to continue . . .
```

Grafos

Exercícios

1) Escreva um método "int `inDegree(int v)`" que calcula e retorna o grau de entrada de um vértice **v** de um grafo dirigido. O método deve ser implementado na classe `TGrafo` da matriz de adjacência. Obs.: Grau de entrada de v é o total de arestas que chegam no vértice v.
2) Escreva o método `outDegree(int v)` que calcula o grau de saída de **v** em grafo dirigido. O método deve ser implementado na classe `TGrafo` que usa matriz de adjacência. Obs.: Grau de saída de v é o total de arestas que saem do vértice v.
3) Fazer o método `degree(int v)` que calcula o grau do vértice de um grafo dirigido. O método deve ser implementado na classe `TGrafo` que usa matriz de adjacência.

Grafos

Exercícios

4) Escreva um método para um grafo direcionado que recebe um vértice como parâmetro e retorne 1 se vértice for uma **fonte** (grau de saída maior que zero e grau de entrada igual a 0), ou 0 caso contrário. O método deve ser implementado para a classe `TGrafo` como matriz de adjacência.

5) Escreva um método para um grafo direcionado que recebe um vértice como parâmetro, retorne 1 se vértice for uma **sorvedouro** (grau de entrada maior que zero e grau de saída igual a 0), ou 0 caso contrário. O método deve ser implementado para a classe `TGrafo` que utiliza matriz de adjacência.

Grafos

Exercícios

6) Escreva um método que receba um **grafo dirigido** como parâmetro e **retorna 1** se o grafo for **simétrico** e 0 caso contrário. O método deve ser implementado para a classe `TGrafo` que utiliza matriz de adjacência.

7) Um **grafo** pode ser armazenado em um arquivo com o seguinte formato:

```
6
8
0 1
0 5
1 0
1 5
2 4
3 1
4 3
3 5
```

Grafos

Exercícios

Onde na primeira linha contém um inteiro **V** (vértice), na segunda contém um inteiro **A** (arestas) e nas demais linha contém dois inteiros pertencentes ao intervalo `0..V-1`. Se interpretarmos cada linha do arquivo como uma **aresta**, podemos dizer que o arquivo define um **grafo** com vértices `0..V-1`. Escreva uma método que receba um nome de arquivo com o formato acima e construa a representação do grafo como matriz de adjacência. (Exemplo de código que manipula arquivo no próximo slide)

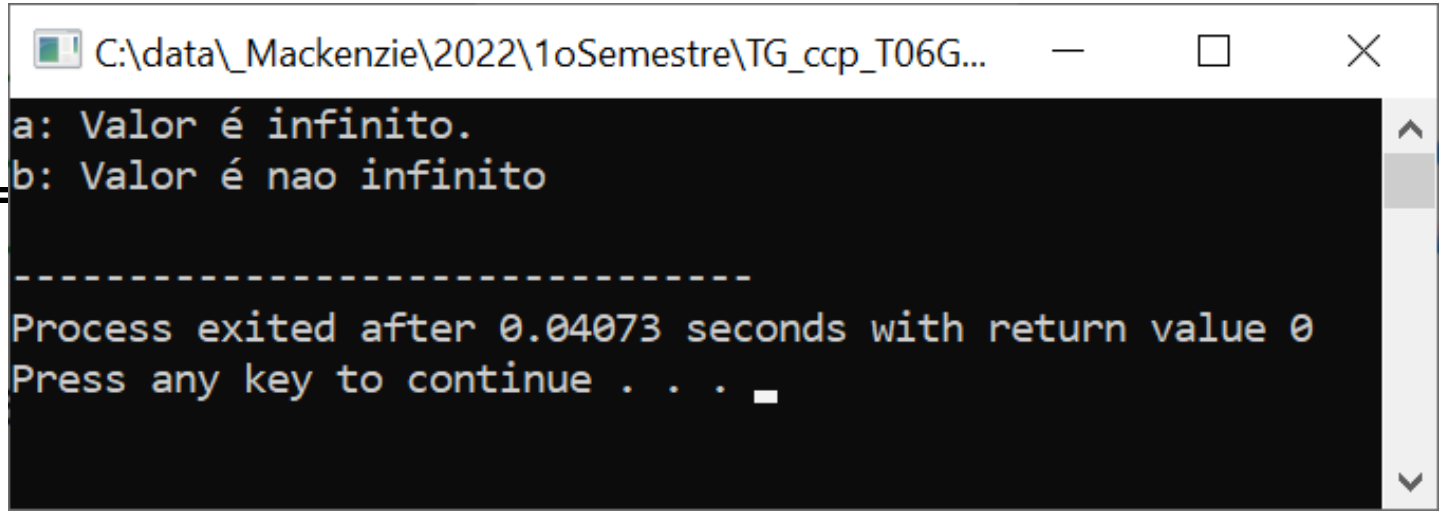
8) Criar uma outra classe TGrafoND e modifique as funções `insereA`, `removeA` e `show` para representar um **grafo não-dirigido** utilizando matriz de adjacência.

9) Fazer o método `degree(int v)` que calcula o grau do vértice de um grafo não-dirigido. O método deve ser implementado na classe `TGrafoND` que usa matriz de adjacência.

10) Modifique a classe TGrafo e os métodos correspondentes para permitir a criação de um grafo direcionado rotulado (valor float) nas arestas. Por exemplio, em C++, para representar o infinito utilize `float a = std::numeric_limits<float>::infinity()`. (Exemplo a seguir).

Grafos

Exercícios




```cpp
#include <iostream>
#include <limits>    // Limites de valores numéricos
#include <locale.h>  // vamos usar para remover a acentação em C++
using namespace std;
int main(){

    setlocale(LC_ALL, "Portuguese");

    float a = std::numeric_limits<float>::infinity();

    if (a == std::numeric_limits<float>::infinity()) cout << "a: Valor é infinito." << endl;
    else cout << "a: Valor é não infinito" << endl;

    float b = -1.2f;

    if (b == std::numeric_limits<float>::infinity()) cout << "b: Valor é infinito." << endl;
    else cout << "b: Valor é não infinito" << endl;
}
```

Grafos

Exercícios

11) Fazer um método que permita remover um vértice do Grafo (não dirigido e dirigido). Não se esqueça de remover as arestas associadas.

12) Fazer um método que verifique e retorne se o grafo (não dirigido) é completo.

13) Fazer um método que verifique e retorne e o grafo (dirigido) é completo.

14) Fazer um método que retorne o complemento (grafo complementar) de um grafo (dirigido ou não) na forma de uma matriz de adjacência.

15) Fazer um método que retorne o tipo de conexidade de um grafo não direcionado (0 - conexo ou 1 - não conexo – desconexo).

16) Fazer um método que retorne a categoria de conexidade para um grafo direcionado (3 – C3, 2 – C2, 1 – C1 ou 0 – c0).

17) Faze um método que retorne o grafo reduzido de um grafo direcionado no formato de uma matriz de adjacência.

Grafos

Exemplo de manipulação de Arquivo – Escrever/Salvar em C++

a) Escrever dados em um arquivo texto.

```
#include <fstream>
using namespace std;
int main(){
      ofstream fout("Teste.txt");
      fout << "Exemplo 1: salvando em arquivo" <<
endl;
      fout << "Pratique!" << endl;
      return 0;
}
```

Grafos

Exemplo de manipulação de Arquivo – Escrita em C++

b) Ler dados de um arquivo texto.

```
#include <fstream>
#include <iostream>
using namespace std;

const int MAX = 80;

int main(){
      char buffer[MAX];

      ifstream fin("Teste.txt");

      while (!fin.eof()){
            fin.getline(buffer, MAX);
            cout << buffer << endl;
      }
      return 0;
}
```

Grafos - Representação - Lista de Adjacência (1/3)

- É uma **técnica** normalmente **utilizada** para representar um **grafo esparso**.

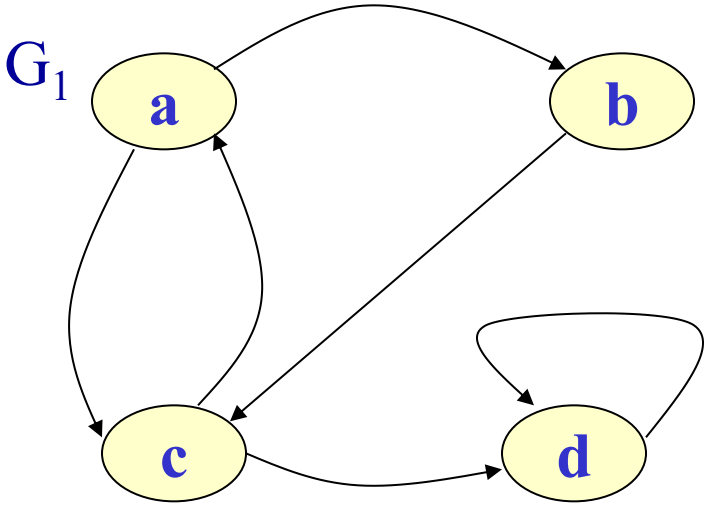
- Um grafo **G** = (V, E) **utiliza | V | listas encadeadas**, uma **para cada vértice**.

- A **lista encadeada** do vértice $\mathbf{v_i} \in \mathbf{V}$ contém os elementos do **conjunto de nós adjacentes a $\mathbf{v_i}$**, ou seja, **{ w: ($\mathbf{v_i}$, w) $\in$ S($\mathbf{v_i}$) }.**

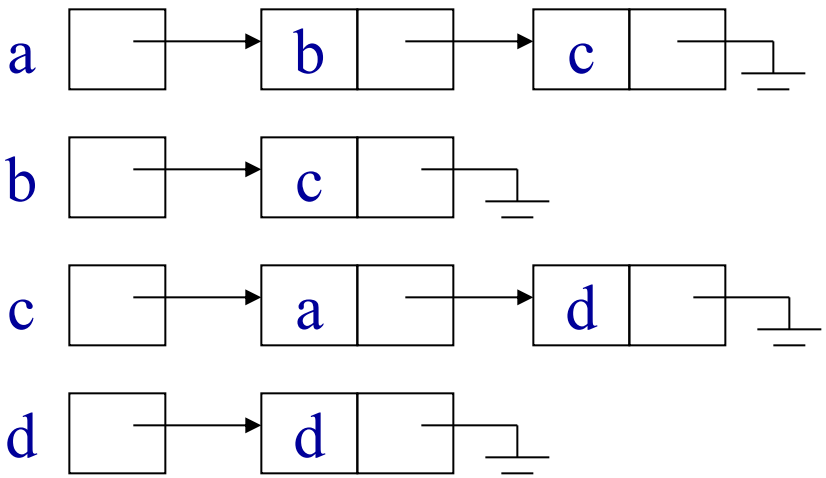
- Essas listas são chamadas **listas de adjacência**.

Grafos - Representação - Lista de Adjacência (2/3)

Exemplo:

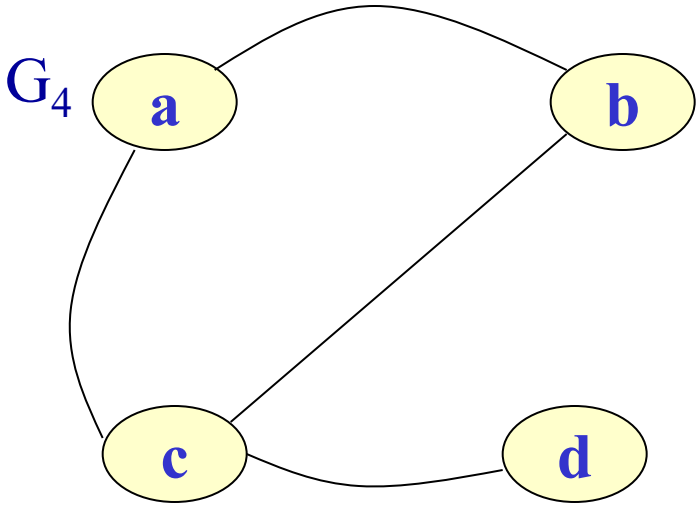


G_1

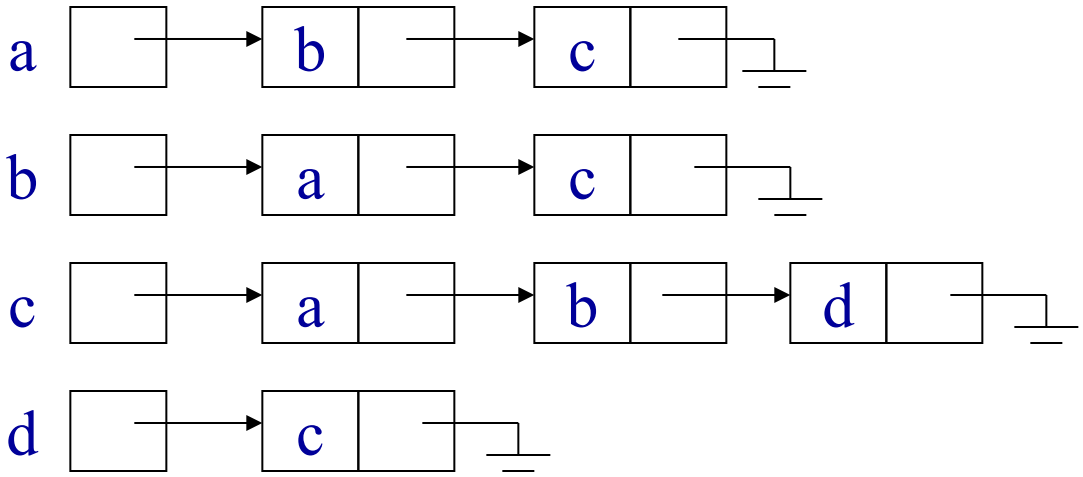


Grafos - Representação - Lista de Adjacência (3/3)

Exemplo:



G_4

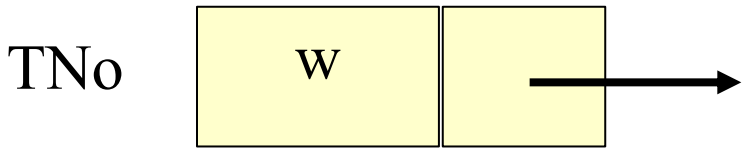


Grafos

Implementação – Grafo Direcionado - Lista de Adjacência – Classe Nó

- Para representar um grafo como uma lista de adjacência vamos definir uma classe com os seus atributos e métodos, como a seguir:

```
// definicao da classe de nós da lista
class TNo{
      public:
      // vértice que é adjacente ao elemento da lista
            int w;
            TNo *prox;
};
```



Grafos

Implementação – Grafo Direcionado - Lista de Adjacência - TGrafo*

- Para representar um grafo como uma matriz de adjacência vamos definir uma classe com os seus atributos e métodos, como a seguir:

```
// definição de uma classe para armezanar um grafo
class TGrafo{
      private:
            int n; // quantidade de vértices
            int m; // quantidade de arestas
      // um vetor onde cada entrada guarda o inicio de uma lista
            TNo **adj;
      public:
            TGrafo( int n );
            void insereA( int v, int w);
            void removeA( int v, int w);
            void show();
            ~TGrafo();
};
```

Baseado na implementação do prof. Ms. Fábio Lucacheski

Grafos

Implementação – Grafo Direcionado - Lista de Adjacência - TGrafo*

```
// Construtor do grafo com a lista de adjacência
TGrafo::TGrafo( int n ){
    // aloca a estrutura TGrafo
    this->n = n;
    this->m = 0;
    // aloca m vetor para guardar lista de adjacencias
    TNo **adjac = new TNo*[n];
    // Inicia o vetor com NULL
    for(int i = 0; i< n; i++)
      adjac[i]=nullptr;
    adj = adjac;
}
```

Baseado na implementação do prof. Ms. Fábio Lucacheski

Grafos

Implementação – Grafo Direcionado - Lista de Adjacência - TGrafo*

```
/* Método que cria uma aresta v-w no grafo. O método supõe
que v e w são distintos, positivos e menores que V. Se o
grafo já tem a aresta v-w, o método não faz nada. O método
também atualiza a quantidade de arestas no grafo. */
void TGrafo::insereA( int v, int w){
    TNo *novoNo;
    // anda na lista para chegar ao final
    TNo *no = adj[v];
    TNo *ant = nullptr;
    // anda na lista enquanto no != NULL E w  > no->w
    while( no && w >= no->w ){
        if( w == no->w)
            return;
        ant = no;
        no = no->prox;
    };
```

Baseado na implementação do prof. Ms. Fábio Lucacheski

Grafos

Implementação – Grafo Direcionado - Lista de Adjacência - TGrafo*

```
    // cria o novo No para guardar w
    novoNo = new TNo;
    novoNo->w = w;
    novoNo->prox = no;
    // atualiza a lista
    if( ant == nullptr){
        // insere no inicio
        adj[v] = novoNo;
    } else
        // insere no final
        ant->prox = novoNo;
    m++;
}
```

Baseado na implementação do prof. Ms. Fábio Lucacheski

Grafos

Implementação – Grafo Direcionado - Lista de Adjacência - TGrafo*

```
/* Método que libera a memória utilizada pela
lista de adjacência. */
TGrafo::~TGrafo(){
    // libera cada um dos nos da lista encadeada
    for( int i = 0; i < n; i++){
        TNo *no = adj[i];
        TNo *ant = nullptr;
        while(no){
            ant = no;
            no = no->prox;
            delete ant;         }    }
    n = 0;
    m = 0;
    // libera o vetor da lista encadeada
    delete [] adj;
};
```

Baseado na implementação do prof. Ms. Fábio Lucacheski

Grafos

Implementação – Grafo Direcionado - Lista de Adjacência - TGrafo*

```
/* Método que remove do grafo a aresta que tem ponta
inicial v e ponta final w. O método supõe que v e w são
distintos, positivos e menores que V. Se não existe a
aresta v-w, o método não faz nada. O método também
atualiza a quantidade de arestas no grafo. */
void TGrafo::removeA(int v, int w){
    // Obtém o início da lista do vértice v
    TNo *no = adj[v];
    TNo *ant = nullptr;
    // Percorre a lista do vértice v
    // procurando w (se adjacente)
    while( no != nullptr && no->w != w ){
            ant = no;
            no = no->prox;
    }
```

Baseado na implementação do prof. Ms. Fábio Lucacheski

Grafos

Implementação – Grafo Direcionado - Lista de Adjacência - TGrafo*

```
    // Se w é adjacente, remove da lista de v
    if (no != nullptr){
      ant->prox = no->prox;
      delete no;
      no = nullptr;
      m--;
      }
}
```

Grafos

Implementação – Grafo Direcionado - Lista de Adjacência - TGrafo*

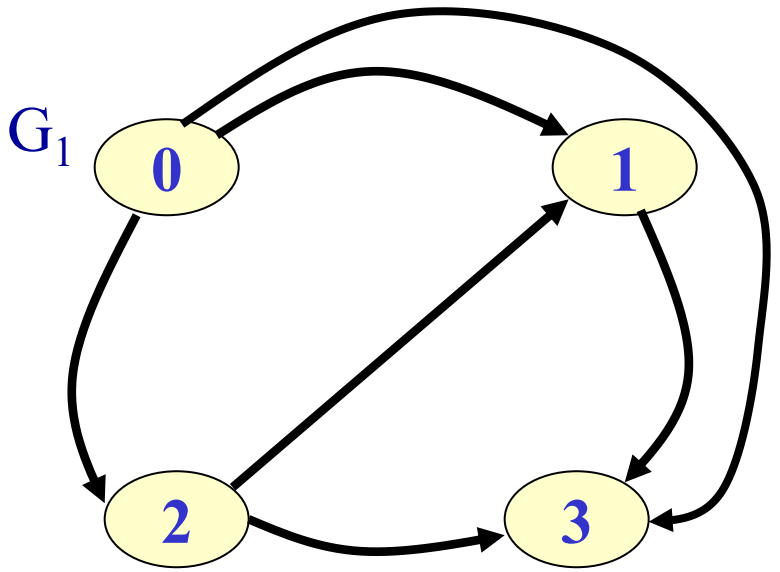
```
/* Para cada vértice v do grafo, este método imprime, em
uma linha, todos os vértices adjacentes ao vértice v
(vizinhos ao vértice v). */
void TGrafo::show(){
    cout << "n: " << n;
    cout << "\nm: " << m;
    for( int i=0; i < n; i++){
        cout << "\n" << i << ": ";
        // Percorre a lista na posição i do vetor
        TNo *no = adj[i];
        while( no ){
            cout << no->w << " ";
            no = no->prox;
        }    }
     cout << endl << "fim da impressao do grafo." << endl;
}
```

Baseado na implementação do prof. Ms. Fábio Lucacheski

Grafos

Implementação – Grafo Direcionado - Lista de Adjacência - TGrafo*

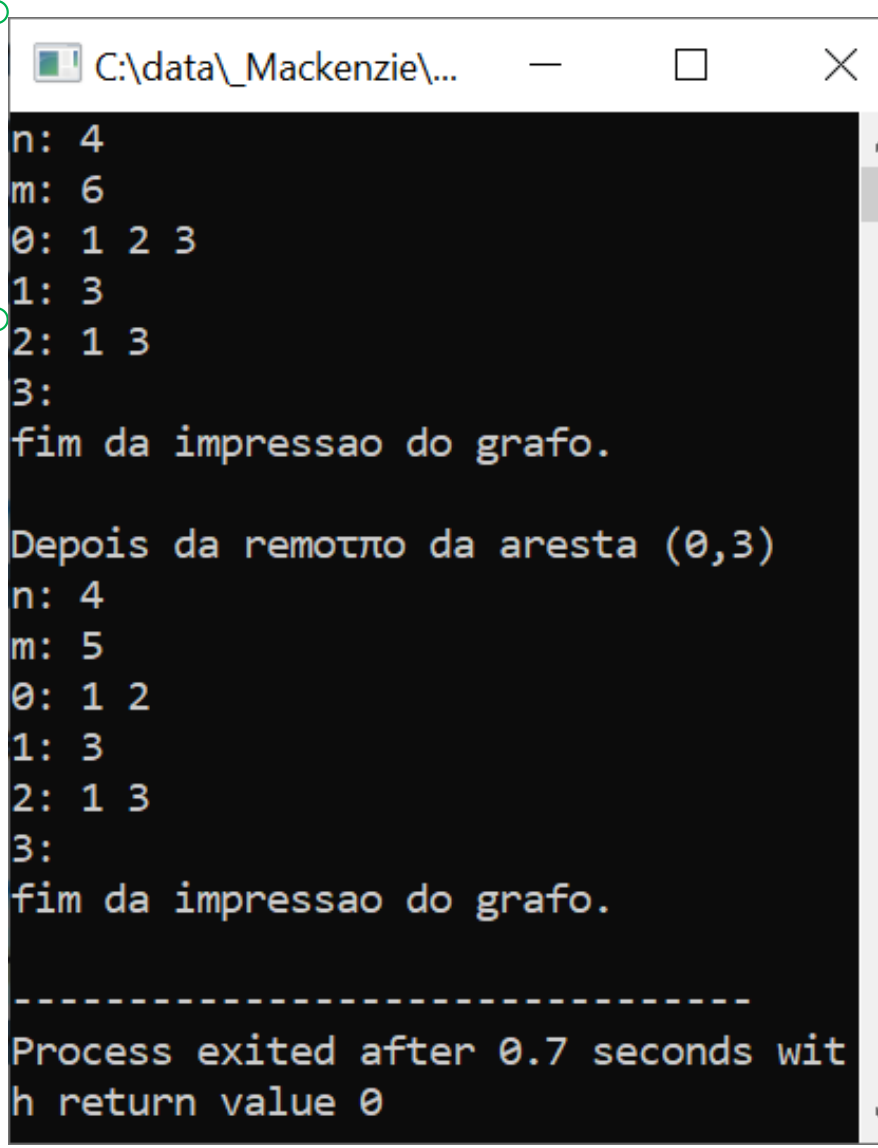
```
int main(){
    // Cria um objeto para um Grafo com 4 vértices
    // Armazendo em uma lista de adjacência
    TGrafo g(4);
    //insere as arestas do grafo
    //A={(0,1),(0,2),(0,3),(2,1),(2,3),(1,3)}
    g.insereA(0,1);
    g.insereA(0,2);
    g.insereA(0,3);
    g.insereA(2,1);
    g.insereA(2,3);
    g.insereA(1,3);
```



Grafos

Implementação – Grafo Direcionado - Lista de Adjacência - TGrafo*

```
    // mostra o grafo preenchido
    g.show();
    // remove a aresta (0,3)
    g.removeA(0,3);
    // mostra o grafo preenchido
    cout << "\nDepois da
    remoção da aresta (0,3)\n";
    g.show();
    return 0;
}
```



Grafos

Exercícios

18) Escreva um método “int inDegree(int v)” que calcula e retorna o grau de entrada de um vértice v de um grafo dirigido fazendo uso da lista de adjacência.

19) Escreva o método outDegree(int v) que calcula o grau de saída de v em grafo dirigido. fazendo uso da lista de adjacência.

20) Fazer o método degree(int v) que calcula o grau do vértice de um grafo dirigido fazendo uso da lista de adjacência.

Grafos

Exercícios

21) Escreva um método que decida se dois grafos direcionados são iguais. O método deve ser implementado para a classe `TGrafo` faz uso da lista de adjacência.

22) Escreva um método que converta uma representação de um grafo em outra. Por exemplo, converta um grafo armazenado em matriz de adjacência em uma lista de adjacência.

23) Escreva um método que receba um grafo armazenado em **lista de adjacência** e inverta a lista de adjacência de todos os vértices do grafo. Por exemplo, se os 4 vizinhos de um certo vértice **u** aparecem na lista `adj[u]` na ordem **v**, **w**, **x**, **y**, então depois da aplicação do método a lista deve conter os mesmos vértices na ordem **y**, **x**, **w**, **v**. Obs.: Vizinhos são todos os vértices ligados ao vértice u.

Grafos

Exercícios

24) Escreva um método que receba um grafo e um vértice como parâmetro e retorne 1 se vértice for uma **fonte** (grau de saída maior que zero e grau de entrada igual a 0), ou 0 caso contrário. O método deve ser implementado para a classe `TGrafo` como lista de adjacência.

25) Escreva um método que receba um grafo e um vértice como parâmetro, retorne 1 se vértice for uma **sorvedouro** (grau de entrada maior que zero e grau de saída igual a 0), ou 0 caso contrário. O método deve ser implementado para a classe `TGrafo` que utiliza lista de adjacência.

26) Escreva um método que receba um **grafo dirigido** como parâmetro e **retorna 1** se o grafo for **simétrico** e 0 caso contrário. O método deve ser implementado para a classe `TGrafo` que utiliza lista de adjacência.

Grafos

Exercícios

27) Um **grafo** pode ser armazenado em um arquivo com o seguinte formato:

```
6
8
0 1
0 5
1 0
1 5
2 4
3 1
4 3
3 5
```

Onde na primeira linha contém um inteiro **V** (vértice), na segunda contém um inteiro **A** (arestas) e nas demais linha contém dois inteiros pertencentes ao intervalo `0..V-1`. Se interpretarmos cada linha do arquivo como uma **aresta**, podemos dizer que o arquivo define um **grafo** com vértices `0..V-1`. Escreva uma método que receba um nome de arquivo com o formato acima e construa a representação de lista de adjacência do grafo.

Grafos

Exercícios

28) Fazer um método que permita remover um vértice do Grafo (não dirigido). Não se esqueça de remover as arestas associadas.

29) Fazer um método que permita remover um vértice do Grafo (dirigido). Não se esqueça de remover as arestas associadas.

30) Fazer um método que verifique se o grafo (dirigido ou não) é completo.

Bibliografia Básica

BONDY, A., MURTY, U.S.R. **Graph Theory**. New York: Springer, 2010.

EVEN, S., EVEN, G. **Graph Algorithms**.2.ed. Boston: Cambridge University Press, 2011.

DIESTEL, R. **Graph theory**. 4.ed. New York: Springer, 2010.